ABSTRACT

The Smart Bus Tracking and Arrival Prediction System is an embedded Internet of Things (IoT) prototype developed to improve the availability of bus-location and arrival information for passengers. The system integrates an ESP32 development board, a NEO-6M GPS receiver and a 16 x 2 I2C liquid crystal display with a locally hosted, browser-based dashboard known as BUSLYTICS. The GPS receiver supplies position, speed, direction and satellite-related information. The ESP32 interprets these measurements, updates the LCD, monitors configured bus stops and publishes telemetry through a local Wi-Fi network.

The BUSLYTICS web application provides an overview of vehicle status, a live-location map, configurable routes and stops, estimated times of arrival, device diagnostics and trip history. For classroom demonstrations, an explicitly labelled simulation mode animates a virtual bus journey and permits adjustment of speed and playback rate without requiring the hardware to be transported. Route settings and compact trip summaries are stored in the ESP32 nonvolatile memory, allowing operation without a dedicated cloud subscription or separate application server.

Real-mode arrival estimates use the great-circle distance to the next stop, configured inter-stop distances and a moving average of GPS speed; a fallback speed is applied when recent measured motion is unavailable. The implementation intentionally operates through an ESP32 access point or the same local Wi-Fi network. It does not currently provide public internet tracking, live traffic prediction or a physical bus-stop display. The completed prototype demonstrates practical interfacing, GPS parsing, network communication, mathematical estimation, embedded storage and responsive web visualization for an educational transport-monitoring application.

Keywords: ESP32, GPS, NEO-6M, IoT, Bus Tracking, ETA, Local Web Server, I2C LCD, Route Simulation, Nonvolatile Storage.

CONTENTS

CHAPTER 1 INTRODUCTION

CHAPTER 2 SYSTEM DESIGN AND OPERATION

CHAPTER 3 HARDWARE DESIGN AND DESCRIPTION

CHAPTER 4 HARDWARE IMPLEMENTATION

CHAPTER 5 SOFTWARE DESCRIPTION

CHAPTER 6 SOFTWARE IMPLEMENTATION

CHAPTER 7 EXPERIMENTAL RESULTS AND DISCUSSION

CHAPTER 8 ADVANTAGES, LIMITATIONS AND APPLICATIONS

CHAPTER 9 CONCLUSION AND FUTURE SCOPE

REFERENCES

APPENDIX A - HARDWARE CONNECTIONS

APPENDIX B - USER OPERATING PROCEDURE

APPENDIX C - SYSTEM PARAMETERS AND API SUMMARY

.

CHAPTER 1

INTRODUCTION

1.1 Overview

Public transport is an important part of daily travel for students, employees and other passengers. Bus schedules provide a useful approximate timetable, but they cannot always tell a waiting passenger where a bus is at a particular moment. Changes in road conditions, congestion, delays at stops and variations in journey duration make fixed schedules insufficient for reliable arrival information. A location-aware system can reduce this uncertainty by presenting the bus position and estimated time required to reach the next selected stop.

The project titled Smart Bus Tracking and Arrival Prediction System addresses this requirement using low-cost embedded hardware and a browser-accessible user interface. A GPS module receives satellite-based positioning data and sends serial NMEA messages to an ESP32 microcontroller. The controller processes the information using the TinyGPSPlus library and publishes the latest device status through a web server. A 16 x 2 character LCD shows essential information directly on the device. A Wi-Fi-connected phone or laptop can open the local dashboard to view the same telemetry in a more detailed graphical form.

The software is branded BUSLYTICS and provides live monitoring, a route editor, stop-wise ETA estimates, hardware diagnostics, a virtual journey simulator and locally stored trip summaries. The complete browser interface is embedded in the ESP32 program. Consequently, a separate server or Firebase project is not required for the finished local version. This architecture is suitable for a college mini-project in which low component cost, practical demonstration and understandable implementation are primary considerations.

1.2 Problem Statement

Passengers waiting at bus stops often lack dependable information about the current position of a bus and its expected arrival. Timetables cannot represent sudden operating delays, and manual status updates are inconvenient. A conventional location-only tracker may show the bus position without answering the more useful question of when the vehicle is likely to reach a selected stop. There is therefore a need for an affordable bus-monitoring prototype that combines GPS position acquisition, configurable routes, simple arrival estimation and an accessible passenger interface.

1.3 Objectives

- To obtain the current geographical coordinates, speed and satellite status from a NEO-6M GPS receiver.
- To interface the GPS receiver with an ESP32 through hardware UART communication.
- To display dashboard connection details and important GPS/route parameters on a 16 x 2 I2C LCD.

- To provide a responsive local web dashboard for monitoring the position and status of one configured bus.
- To create a route configuration facility for bus ID, route name, stop coordinates and fallback speed.
- To estimate arrival times using distance calculations and recent GPS speed information.
- To develop a clearly identifiable virtual-bus simulator for safe indoor demonstrations.
- To retain route settings and a limited history of completed journeys in ESP32 flash memory.
- To expose useful hardware health information and document practical system limitations.

1.4 Literature Survey

The Phase-1 presentation identifies prior work on college bus tracking, fuel monitoring, school bus tracking and arrival prediction in Indian transport conditions. These studies provide context for the choice of GPS positioning, electronic communication and estimated arrival information. The following summaries are based on the descriptions contained in the project synopsis; the original papers were not independently evaluated as part of preparing this report.

1.4.1 College Bus Tracking and Notification System

The work attributed to K. Premkumar and co-authors [1] describes tracking college buses and supplying useful information or notifications to passengers. It establishes the relevance of bus location visibility in an educational-transport setting. The present project concentrates on embedded GPS acquisition and a local web interface rather than a notification service.

1.4.2 Design of Bus Tracking and Fuel Monitoring System

The study by S. Vigneshwaran and co-authors [2] combines location-related information with vehicle monitoring. This demonstrates that tracking systems may be extended to additional vehicle parameters. BUSLYTICS currently monitors GPS-derived motion and device health; fuel sensing is outside the implemented project scope.

1.4.3 IoT-Based School Bus Tracking and Arrival-Time Prediction

The work listed for Jisha R. C. and co-authors [3] uses GPS and communication technologies to support bus monitoring and anticipated arrival information. The relevance to the present design lies in connecting positioning data with a passenger-facing interface. BUSLYTICS adopts a low-cost ESP32-based local implementation and a simplified arrival estimation method.

1.4.4 Real-Time Bus Arrival-Time Prediction under Indian Traffic Conditions

The work attributed to B. Dhivyabharathi and co-authors [4] highlights the importance of traffic and operating conditions when predicting bus arrivals. Unlike a traffic-aware prediction system, the implemented BUSLYTICS real GPS mode estimates time from distance and recent or fallback speed. This difference is an important limitation and provides a direction for future improvement.

1.5 Existing and Proposed System Summary

The existing approach in many local transport services relies on printed or digital schedules and manual enquiries. These sources may be helpful for route planning but do not independently establish

the live position of a particular bus. Some systems offer GPS tracking without a transparent estimate for every stop. In the intended proposed architecture of the original synopsis, GPS positions would be transmitted to a cloud server for remote passenger access and ETA processing.

The final implemented project adopts a simpler local-first architecture. An ESP32 collects live GPS information, serves an HTTP dashboard to users on the same network and stores configuration and trip summaries internally. The dashboard calculates approximate ETA values and also provides a simulation mode for demonstrations. Cloud-based tracking and live traffic data remain future extensions rather than completed features.

1.6 Hardware and Software Components Used

Hardware	Software / library
ESP32 DevKit V1	Arduino IDE with Espressif ESP32 board package
NEO-6M GPS receiver	TinyGPSPlus GPS parser
16 x 2 I2C LCD, 0x27	LiquidCrystal_I2C and Wire
Regulated USB power supply	WiFi, WebServer and Preferences
Connecting wires / mounting base	ArduinoJson 7.x
Phone or laptop for viewing	HTML, CSS, JavaScript, Leaflet, OpenStreetMap

1.7 Organization of the Report

Chapter 2 explains the overall architecture and operating principle. Chapter 3 describes the major hardware components. Chapter 4 presents the actual wiring and integration procedure. Chapter 5 discusses the software tools and libraries. Chapter 6 documents the firmware logic, mathematical methods and web application. Chapter 7 records the implementable functions, test methods and observations supported by the source code. Chapter 8 presents advantages, limitations and applications. Chapter 9 provides the conclusion and future development possibilities.

1.8 Summary

The chapter introduced the transportation information problem, project objectives, relevant prior approaches and the implemented system boundaries. The next chapter describes how positioning, embedded processing, local Wi-Fi and passenger visualization work together.

CHAPTER 2

SYSTEM DESIGN AND OPERATION

The system is designed as a compact combination of GPS hardware, microcontroller firmware, local wireless networking and a responsive browser interface. The overall design separates physical GPS acquisition from browser-side presentation and simulation. This separation permits a useful classroom demonstration while preserving access to actual GPS hardware status.

2.1 Block Diagram

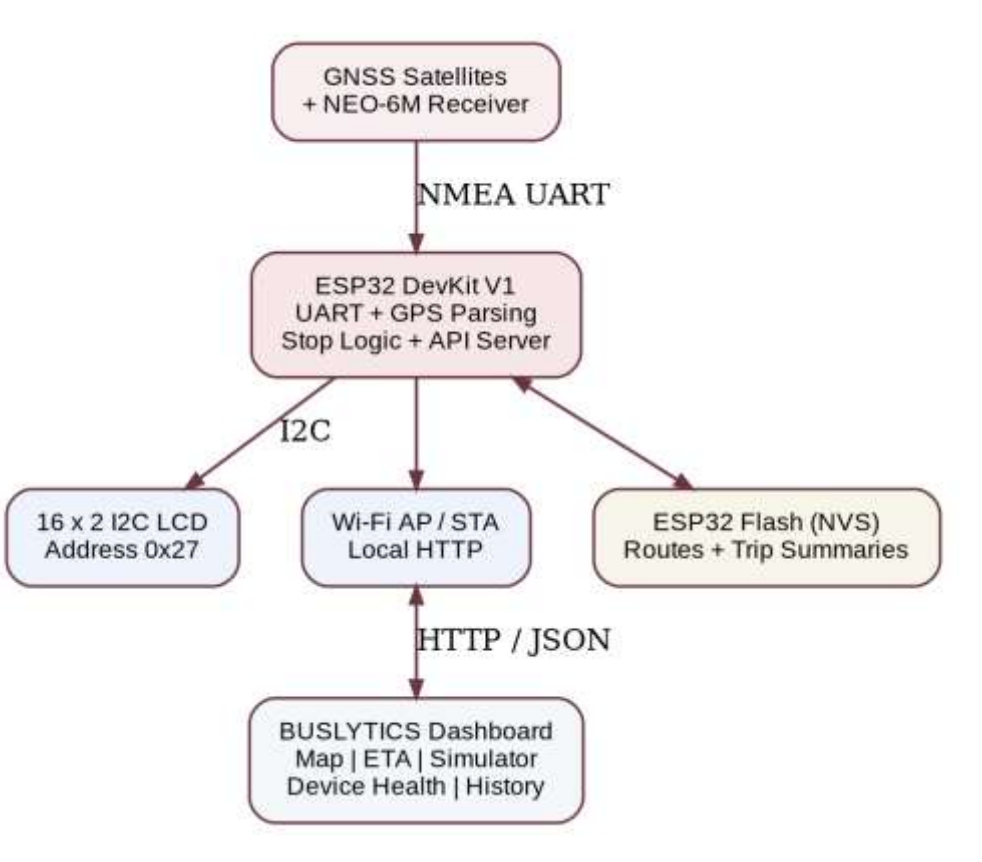


Figure 2.1 Block diagram of the implemented Smart Bus Tracking and Arrival Prediction System

2.2 Working Mechanism

After power is applied, the ESP32 initializes the serial interface for the GPS receiver, starts the I2C LCD, loads previously saved route settings from nonvolatile memory and starts a local Wi-Fi access point. It also attempts connection to a configured Wi-Fi network in station mode. If a router connection is unavailable, the access point remains available for direct monitoring.

The GPS receiver continuously transmits NMEA sentences. The ESP32 passes received characters to TinyGPSPlus, which updates latitude, longitude, speed, course, satellite and other fields when valid messages are received. A location is treated as fresh only when the parser reports valid coordinates whose age is below five seconds. Recent speed measurements are maintained in a small buffer and used by the arrival prediction features.

The ESP32 exposes JSON telemetry through a local HTTP API, and the dashboard requests updated data approximately every 2.5 seconds. The web interface uses the incoming GPS coordinates to position a marker on a map, display speed and satellite information, and generate stop-wise time estimates. Separately, the LCD cycles through information pages after the browser first accesses the dashboard. Route configuration and trip summaries are retained across ordinary restarts in ESP32 flash memory.

2.3 Modes of Operation

2.3.1 Real GPS Mode

Real GPS mode presents measurements obtained from the NEO-6M receiver. When a fresh position fix exists, the dashboard can display coordinates and movement-related values. Device Health remains associated with the physical receiver even when other dashboard features are in use. This separation is important for avoiding confusion between measured and simulated location values.

2.3.2 Demonstration / Simulation Mode

The simulation mode provides a virtual vehicle moving along a configured route. A browser can request road geometry from an external routing service when an internet connection is available. Otherwise, straight-line segments are used. The operator can adjust simulated speed and playback rate. All virtual readings are explicitly labelled as simulated; the simulation is not claimed to represent live GPS measurement.

2.3.3 Local Network Modes

The device uses Wi-Fi AP+STA operation. Station mode allows connection to an existing Wi-Fi router, while AP mode creates a direct hotspot that can be reached using the default local access-point address 192.168.4.1. Both methods provide local access rather than remote public-internet access.

2.4 Software Data Flow

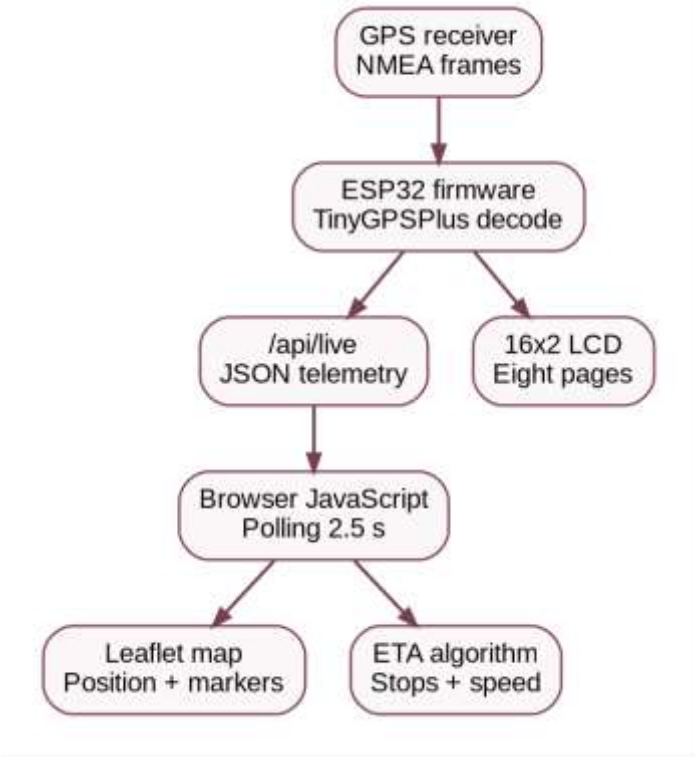


Figure 2.2 Information flow from GPS NMEA messages to local dashboard and LCD

2.5 Functional Blocks and Responsibilities

Block	Primary function
GPS receiver	Collects position, time and velocity measurements from satellite navigation signals.
ESP32 controller	Parses GPS, tracks freshness, manages route progression and responds to requests.
I2C LCD	Displays address, coordinates, speed, bus identity, next-stop ETA and health.
Wi-Fi AP/STA	Connects phones and laptops to the ESP32 local HTTP server.
Dashboard	Presents maps, live telemetry, stops, simulation, configuration and history.
NVS flash	Persists routes and compact history; no cloud database is used.

2.6 System Boundaries

The project is configured for one bus and supports up to twelve ordered stops. The live dashboard relies on local connectivity to the ESP32. Map tiles and the optional road-routing service depend on external internet access. There is no cellular modem, cloud backend, traffic-aware model or standalone roadside passenger board in the completed configuration.

CHAPTER 3

HARDWARE DESIGN AND DESCRIPTION

The implemented hardware has deliberately few components. The controller is responsible for both real-time embedded functions and the web server, while the GPS module provides the physical location input. The LCD is the local output element and provides an alternative way to confirm that the board is active.

3.1 ESP32 Development Board

The ESP32 DevKit V1 is the primary processing and communication device. It provides programmable digital pins, hardware serial interfaces, I2C support, flash memory and integrated Wi-Fi. These resources allow a single controller to acquire NMEA data, update an LCD, maintain stored route information and serve a browser application. The implementation uses UART2 for GPS, I2C for the character display and the built-in Wi-Fi stack for local HTTP access.

In the firmware, a WebServer object manages requests on port 80. Preferences provides an interface to nonvolatile storage, while WiFi supports AP+STA operating mode. Free-heap reporting and uptime counters are used to assess the state of the controller during operation. The exact available memory and electrical limits depend on the development-board variant.

3.2 NEO-6M GPS Receiver

The NEO-6M module is the satellite-navigation interface. Once appropriate signals are acquired, it provides NMEA records from which position and speed can be parsed. The implemented connection uses the module TX output to ESP32 GPIO16, the RX input of hardware Serial2. The module RX connection is not required for this receive-only application. The firmware configures GPS serial communication at 9600 baud.

GPS readings can vary due to buildings, obstruction, antenna placement and multipath reflections. A valid serial message does not always mean that fresh usable coordinates exist; consequently, the software checks validity and age. A good outdoor antenna position is recommended for testing.

3.3 16 x 2 I2C LCD Display

A 16-column, two-row character LCD is used for immediate feedback without requiring a phone. Its I2C backpack reduces the number of controller connections to SDA, SCL, power and ground. The configured address is 0x27, with GPIO21 and GPIO22 assigned to SDA and SCL. The LCD presents a dashboard IP address until the web page has been opened; thereafter, it rotates through eight status pages at approximately 3.5-second intervals.

The display pages include real GPS status, speed, coordinates, configured bus and route, next stop and approximate ETA, satellite/HDOP status, Wi-Fi details and controller health. If authenticated

simulator mirroring is enabled, appropriately labelled simulated information can be shown briefly. Ordinary simulation in the browser does not replace the physical GPS status.

3.4 Power Supply and Voltage Compatibility

The ESP32 requires a stable supply through a suitable USB or board power connection. The GPS receiver must be powered in accordance with the voltage rating of the particular module, and all signal devices must share a common ground. The LCD backpack must not pull the ESP32 I2C lines directly to 5V unless the interface is explicitly level compatible. A bidirectional level shifter may be necessary for a 5V I2C backpack.

USB power is adequate for a bench demonstration when it meets the current requirements of the ESP32 and connected modules. For mounting inside a moving vehicle, the system would need a vehicle-safe regulated power converter, surge protection, proper cable strain relief and enclosure planning. These measures are recommended for future deployment but are not verified as part of the supplied prototype.

3.5 User Device and Wireless Communication

A smartphone or laptop acts as the web-dashboard viewing device. The project does not require a dedicated touchscreen on the microcontroller. The ESP32 local access point and/or an existing Wi-Fi router provide access to the embedded web application. When external internet access is available, the browser may additionally load map tiles and road-routing information from online services.

3.6 Hardware Requirements

No.	Component	Quantity	Purpose
1	ESP32 DevKit V1	1	Controller, Wi-Fi and HTTP server
2	NEO-6M GPS with antenna	1	Latitude, longitude, speed and satellites
3	16 x 2 LCD with I2C backpack	1	Local user feedback
4	USB power source / compatible cable	1	Regulated power to controller
5	Jumper wires / connection board	As required	Electrical interconnection
6	Enclosure or demonstration base	1	Mechanical mounting
7	Phone/laptop	1	Dashboard viewing and setup

3.7 Summary

The selected hardware is sufficient for local GPS tracking, LCD feedback and browser-based display. No GSM module, Firebase server or physical bus-stop LED board forms part of the implemented setup.

CHAPTER 4

HARDWARE IMPLEMENTATION

The hardware implementation focuses on secure power distribution, correct serial reception from the GPS module and dependable I2C communication with the LCD. The circuit is compact, making it suitable for mounting on a project demonstration base.

4.1 Circuit Diagram



Figure 4.1 Electrical connection diagram for ESP32, NEO-6M GPS and I2C LCD

GPS and LCD supply pins must be connected according to the exact module ratings. When using a 5V LCD backpack, check I2C logic levels before connecting directly to ESP32 GPIO21/22.

4.2 Connection Table

Module	Signal	ESP32 connection / note
NEO-6M GPS	TX	GPIO16 (UART2 RX)
NEO-6M GPS	RX	Not required for the receive-only configuration
NEO-6M GPS	GND	Common GND
NEO-6M GPS	VCC	Verified module-rated supply
16 x 2 I2C LCD	SDA	GPIO21 (voltage compatible)
16 x 2 I2C LCD	SCL	GPIO22 (voltage compatible)
16 x 2 I2C LCD	GND	Common GND
16 x 2 I2C LCD	VCC	LCD-rated supply; level shifting if necessary
ESP32	Power	Compatible regulated USB/board input

4.3 Hardware Assembly Procedure

1. Mount the ESP32, GPS receiver and LCD so that connector pins are accessible and no conductive part can create a short circuit.
2. Connect common ground between all devices before testing signal connections.
3. Connect GPS TX to GPIO16 and provide the GPS module with its rated supply; leave GPS RX unconnected for receive-only use.
4. Connect LCD SDA and SCL to GPIO21 and GPIO22 through a voltage-compatible I2C arrangement.
5. Power the ESP32 using a stable source and check for unexpected reset messages.
6. Upload the firmware and verify GPS serial reception, LCD initialization and the dashboard connection address.
7. Move the GPS antenna to a location with a suitable view of the sky and confirm the fresh-fix indication.

4.4 Initialization and Network Connectivity

At startup, the program initializes Serial, starts the GPS UART, configures Wire for the I2C pins, initializes the LCD and loads configuration information from NVS. The ESP32 then enables Wi-Fi AP+STA, starts its own access point and attempts to join the configured router. It initializes the HTTP routes and begins accepting requests. A phone can join the local hotspot and open 192.168.4.1 in a browser.

4.5 Integration Considerations

The GPS module should not be powered by an arbitrary GPIO output; power must come from a suitable supply. Signal and supply wiring should be checked before power is applied. The LCD may require a level shifter, especially if its I2C pull-ups are tied to 5V. Keep GPS antenna placement clear of shielding or dense wiring. Since the web server uses the same controller as the GPS parser, the software loop must continue to service serial input and HTTP requests regularly.

4.6 Summary

The integrated circuit uses two data interfaces: UART from the GPS module and I2C to the display. Wi-Fi is integrated into the ESP32, removing the need for an external communication board in the local prototype.

CHAPTER 5

SOFTWARE DESCRIPTION

The project software combines C++ embedded programming and a browser-based user interface. Both the HTTP server and the webpage are distributed as a single ESP32 sketch. This section describes the tools, libraries, storage method and technologies used to build the system.

5.1 Arduino Integrated Development Environment

Arduino IDE is used to edit, compile and upload the ESP32 sketch. The Espressif ESP32 board package provides the microcontroller support, Wi-Fi interfaces and board-specific libraries. After selecting the appropriate ESP32 board and serial port, the program can be compiled and transferred over USB. The Serial Monitor is used to inspect initialization, Wi-Fi changes and periodic GPS status reports.

5.2 Libraries Used

Library	Role in project
WiFi.h	Connects to router and creates the local access point.
WebServer.h	Handles browser and JSON API requests on port 80.
Preferences.h	Saves route configuration and trip summaries in flash NVS.
Wire.h	Controls the ESP32 I2C interface for the LCD.
TinyGPSPlus.h	Parses NMEA location, speed and satellite-related data.
LiquidCrystal_I2C.h	Updates the 16 x 2 character LCD.
ArduinoJson.h (7.x)	Serializes and validates route and telemetry JSON.
math.h	Supports distance, trigonometric and numeric checks.

5.3 Embedded Web Dashboard

BUSLYTICS uses HTML to organize pages, CSS for the responsive visual presentation and JavaScript for interactions. The dashboard contains Overview, Live Tracking, Bus Simulator, Stops & ETA, Device Health, Trip History and Route Setup. It fetches data from the ESP32 through relative HTTP endpoints, which means it normally works without needing a separately hosted frontend. The integrated page is kept in program memory as a PROGMEM HTML string.

The visual interface uses Leaflet for maps and OpenStreetMap as a source of map tiles. The optional simulator obtains road geometry from an online route service, but falls back to direct stop segments if it is unreachable. The external map assets are not stored offline in the ESP32 sketch. As a result, location coordinates and other text telemetry remain available locally even when map tiles cannot load.

5.4 JSON Communication

The firmware provides the latest GPS and controller status in a structured JSON object. The browser polls /api/live approximately every 2.5 seconds. Important fields include the bus ID, route name, next stop index, uptime, free heap, Wi-Fi status and GPS object. The GPS object contains a validity flag, position age, coordinates, satellites, speed, moving-average speed, course and HDOP where available.

Configuration information is retrieved through /api/config and can be changed through an authenticated POST request. Separate API handlers support trip history, manually started GPS-trip recording, simulated-trip logging and optionally mirrored simulation details on the LCD. Data validation rejects invalid coordinates, excessive stop counts and other unsuitable requests.

5.5 Nonvolatile Storage

The Preferences library saves route information as JSON in an ESP32 NVS namespace. Bus identity, route title, default speed, next-stop index and up to twelve stops can persist across normal restarts. A separate namespace stores up to thirty trip summaries. The two namespaces prevent clearing journey history from automatically removing the route configuration.

5.6 Access Control and Security

The dashboard requires an administrator password to modify the configured route and undertake certain trip-history operations. A password is submitted in the X-Admin-Key header. This is useful as a basic access check within a trusted local demonstration network. However, the HTTP connection is not encrypted, and telemetry endpoints are not private to a particular passenger. The prototype is therefore unsuitable for public deployment without strengthened authentication, encryption and privacy safeguards.

5.7 Software Upload and Operation

8. Install the Arduino IDE and the ESP32 board package.
9. Install compatible TinyGPSPlus, LiquidCrystal_I2C and ArduinoJson 7.x libraries.
10. Select the matching ESP32 development board and USB COM port.
11. Enter locally appropriate Wi-Fi and administrator credentials privately in the configuration section of the sketch.
12. Compile and upload the firmware to the ESP32.
13. Open the Serial Monitor and check the access-point address, network state and GPS parsing.
14. Connect a browser to the same network, open the displayed IP and configure a route.

CHAPTER 6

SOFTWARE IMPLEMENTATION

The application is divided into embedded acquisition and control routines, JSON API handlers and browser-side dashboard logic. The physical GPS remains the authoritative sensor source. Simulated data is generated in the browser and clearly separated from measured GPS values.

6.1 Program Flowchart

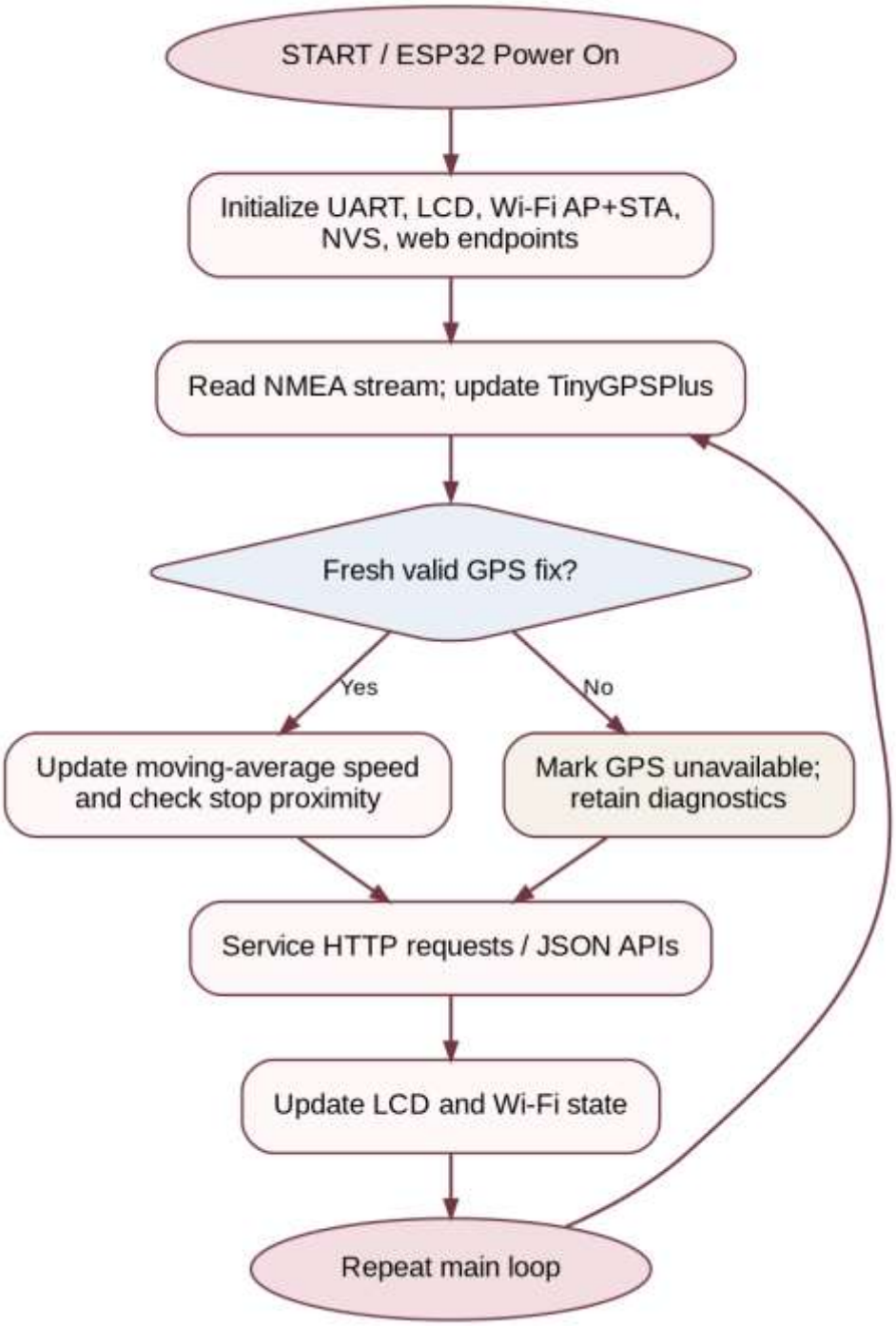


Figure 6.1 Main firmware control flow

6.2 Main Firmware Algorithm

15. Initialize serial debugging, UART2 GPS reception, I2C LCD and nonvolatile storage.
16. Read stored bus identity, route name, stop list, next stop index and fallback speed.
17. Enable the ESP32 access point and attempt station Wi-Fi connection.
18. Register HTTP routes and start the local web server.
19. Read incoming GPS bytes and update the TinyGPSPlus parser.
20. Update recent average speed at the defined sampling interval and check the freshness of the GPS fix.
21. When a valid GPS fix exists, check proximity to the next selected stop and advance the stop index only after sustained presence.
22. Service HTTP requests, update LCD information pages and retry station Wi-Fi when necessary.
23. Repeat continuously while the controller is powered.

6.3 GPS Validity and Moving-Average Speed

A coordinate fix is considered fresh when gps.location.isValid() is true and the age of that location is less than 5,000 milliseconds. Invalid or stale fixes are not presented as current latitude and longitude. The firmware samples speed at intervals of about two seconds, constrains each accepted sample to a configured maximum and stores up to ten recent values in a circular buffer.

The average speed is determined from the accepted readings:

$$v_avg = (v1 + v2 + ... + vn) / n, \text{ where } 1 <= n <= 10$$

If GPS or speed values are stale, the buffer is reset and the mean-speed value becomes zero. This is a simple low-resource smoothing method; it is not a statistical prediction model.

6.4 Distance Computation

The firmware and browser use a great-circle (Haversine) computation to estimate the distance between two latitude/longitude points. For Earth radius R = 6371 km, latitude difference Delta(phi) and longitude difference Delta(lambda), the relationship is:

$$a = \sin^2(\Delta\varphi/2) + \cos(\varphi_1)\cos(\varphi_2)\sin^2(\Delta\lambda/2)$$

$$d = 2R \operatorname{atan2}(\sqrt{a}, \sqrt{1-a})$$

The result is the shortest surface distance between the points, not necessarily the length of the road followed by the bus. Road curvature, diversions and junctions can make actual travel distance longer.

6.5 Estimated Time of Arrival

The dashboard calculates an ETA for the next stop based on the current GPS position and the selected stop coordinate. Subsequent stop estimates add straight-line distances between consecutive configured stops. The speed used for real GPS prediction is the moving-average GPS speed when it is at least 2 km/h; otherwise, the configured fallback speed is used. The default fallback is 25 km/h.

ETA (minutes) = ceil[Remaining distance (km) / Speed (km/h) x 60]

For example, a bus estimated to be 5.0 km from a stop and moving at 30 km/h has ETA = 5/30 x 60 = 10 minutes. This is a mathematical illustration rather than a field measurement. The estimate does not directly incorporate traffic, dwell time at stops or a road-network model in the real GPS mode.

6.6 Automatic Next-Stop Advancement

The controller compares its current coordinates with the next saved stop. If the calculated straight-line separation is 80 metres or less, an arrival timer begins. The stop index advances only after the bus remains continuously inside this zone for at least eight seconds. Leaving the zone before the threshold resets the timer. After advancement, the new index is written to flash. This method reduces false triggers from brief noisy position readings, but high vehicle speed or GPS error can still affect detection.

6.7 Bus Journey Simulation

The simulator allows the operator to select a saved route with at least two stops. Where internet connectivity permits, the browser requests driving-route geometry and animates a bus marker along the returned polyline. If road routing is unavailable, the bus follows direct segments between stops. The operator can choose a simulated speed from 10 to 100 km/h and accelerate the demonstration playback by factors such as 10x, 30x, 60x or 120x.

The virtual position, simulated travel duration and predicted stop arrival values are calculated independently of the physical GPS feed. The interface highlights that the values are simulated. Optional authenticated mirroring to the LCD is temporary and labelled SIM, while physical device-health readings remain based on the actual ESP32 and GPS.

6.8 Trip-History Implementation

A real trip begins when the operator sends the authenticated start command. The controller then accumulates selected valid movement increments and summary speed statistics until an authenticated finish command is received. The implementation accepts successive valid GPS movement steps within an 8-to-95-metre range, avoiding some small jitter and unusually large position jumps. This also means very small movements may not be counted.

Simulation records are created from virtual route and clock values. The system stores a compact record containing the journey type, status, route, bus ID, start and finish time, distance, duration and speed statistics. A rolling capacity of thirty summaries prevents uncontrolled storage growth. An unfinished real trip may be lost if the ESP32 loses power before it is saved.

6.9 Web Endpoints

Endpoint	Method	Function
/	GET	Serve embedded BUSLYTICS dashboard
/api/live	GET	Latest physical GPS and device status

Endpoint	Method	Function
/api/config	GET	Retrieve bus and route configuration
/api/config	POST	Save authenticated route settings
/api/lcd-sim	POST	Authenticated temporary simulation LCD mirror
/api/trips	GET	Read saved trip summaries
/api/trips/start-gps	POST	Begin a physical GPS trip record
/api/trips/stop-gps	POST	Finish and save a GPS trip
/api/trips/record	POST	Save a simulated-trip summary
/api/trips/delete	POST	Delete one saved trip
/api/trips/clear	POST	Clear all saved trip summaries

6.10 Timings and Operating Parameters

Parameter	Configured value / behavior
GPS freshness threshold	Under 5 seconds
GPS speed sampling	Every 2 seconds
Moving average	Up to 10 samples
Dashboard polling	Approximately every 2.5 seconds
LCD refresh	Approximately every 0.5 seconds
LCD page change	Approximately every 3.5 seconds after browser opens
Geofence radius	80 metres
Minimum dwell for stop advance	8 seconds
Max configured stops	12
Saved trip summary capacity	30
Default fallback speed	25 km/h
Wi-Fi station retry	About every 10 seconds

CHAPTER 7

EXPERIMENTAL RESULTS AND DISCUSSION

The completed source code provides implementation evidence for live GPS telemetry, the LCD information pages, route management, simulation and trip-history recording. The project developer has stated that the integrated project is completed. However, calibrated field measurements, outdoor comparison logs and photographs of the final assembled unit were not supplied with the report materials. Therefore, this chapter documents the available software outcomes and a reproducible verification procedure without inventing measured accuracy values or test photographs.

7.1 Initial Boot and Network Availability

At startup, the firmware creates an ESP32 hotspot and attempts to join its configured Wi-Fi router. The LCD shows a usable dashboard address before the dashboard has been opened. Opening the root webpage causes the LCD to begin cycling through eight information pages. The access-point address remains 192.168.4.1; when a router connection is active, a different station IP can also be used.

7.2 GPS Acquisition and Live Telemetry

After the GPS receiver acquires a fresh fix, the firmware publishes coordinates together with available satellite, speed, heading and HDOP fields. The dashboard presents these measurements on the Overview and Device Health screens. GPS fix validity is derived from the physical receiver, not from the virtual simulator. A dashboard opened before GPS acquisition displays a waiting or searching status.

7.3 Web Dashboard Output

The BUSLYTICS dashboard includes a dark-themed overview, live map, upcoming stops, recent-speed visualization and a tracking snapshot. The Live Tracking page supports marker display, recent position history and a map-centered view. The Stops & ETA page lists stop-wise estimated arrival values and the method notes explaining that the real GPS estimates are approximate.

The browser source provides explicit status labels for real GPS operation, simulation, offline communication and GPS searching. Actual map tiles are loaded from internet-hosted services; lack of an external connection may leave the map unavailable while local telemetry remains functional.

7.4 Virtual Journey Demonstration

A route of two or more stored stops can be loaded into the simulator. The animated marker then moves along available driving geometry or a fallback straight-line route. Simulation controls permit adjustments to movement speed and playback rate. ETA labels, trip progress and virtual arrival states update during the animation. This mode is designed for controlled demonstration and is visibly distinguished from live tracking.

7.5 Flash-Based Route and Trip Records

The application maintains an administrator-controlled route with the vehicle ID, stop coordinates, ordered stop sequence and fallback speed. The settings are retained in nonvolatile flash storage. Trip History displays up to thirty compact summary records with separate SIM and GPS labels, and the user can remove individual records or clear all history. The stored summaries do not constitute a complete historical GPS-coordinate log.

7.6 Functional Test Matrix

Test case	Expected behavior	Evidence / verification status
Boot and LCD	Show dashboard IP and startup status	Implemented in firmware; verify on physical LCD
GPS acquisition	Display fresh fix and measured coordinates	Code supports freshness checks; outdoor log not provided
Live dashboard	Poll JSON and update status/map	Implemented; browser/network testing required
Save route	Reject invalid settings and retain valid stops	Input checks and NVS storage present
ETA computation	Calculate stop-wise distance/speed estimate	Algorithm present; field accuracy not measured
Stop arrival	Advance after <=80 m for 8 s	Algorithm present; road test required
Simulator	Animate route and label readings SIM	Implemented; external routing optional
Trip history	Save/list/delete up to 30 summaries	Storage handlers present; restart test recommended
Network loss	Retain local AP and expose offline state	AP+STA and retry logic present; test required
Security	Reject unauthorized configuration writes	Header password check present; no HTTPS

7.7 Example ETA Calculation

Quantity	Illustrative value
Distance to next stop	5.0 km
Recent average bus speed	30.0 km/h
Calculated ETA	10 minutes
Assumption	Constant speed over direct distance

If the actual road journey is longer than the direct GPS separation, this estimate may be optimistic. If the bus stops moving, the configured fallback speed can produce a nonzero ETA even though the bus is delayed. These are limitations of the algorithm rather than errors in numerical calculation.

7.8 Discussion of Results

The software design successfully brings together GPS acquisition, LCD status display, HTTP telemetry and an interactive local web interface in one microcontroller program. This supports the educational goal of demonstrating integration across sensing, embedded software and user

visualization. The simulator is especially valuable for explaining stop progression without requiring continuous movement of the hardware.

Quantitative claims such as positioning accuracy, mean ETA error, cellular coverage, battery runtime or percentage reliability cannot be reported because the necessary measurements have not been provided. A complete field evaluation would record GPS positions and predicted ETA alongside actual stop arrival times over several routes and operating conditions.

7.9 Suggested Evidence for Final Submission

- Photograph of the physically assembled ESP32, NEO-6M and LCD wiring.
- Outdoor GPS receiver screenshot or serial log showing a fresh satellite fix.
- Overview and Live Tracking browser screenshots from the physical ESP32 IP.
- A configured route with the actual bus-stop names and coordinates.
- A before/after sequence showing the simulator and clear SIM labels.
- Trip History page showing a saved journey, with private details removed.
- A short field-test table comparing actual arrival time to predicted ETA.

CHAPTER 8

ADVANTAGES, LIMITATIONS AND APPLICATIONS

8.1 Introduction

The prototype aims to provide passenger-oriented bus information with inexpensive hardware and accessible software. Its main strengths are compact integration and demonstration value, while its limitations arise from local network range, GPS uncertainty, simplified arrival estimation and limited production security.

8.2 Advantages

- Low component count: one ESP32 provides computation, networking and local HTTP services.
- Current coordinates and motion information can be viewed without manually reading serial output.
- A local browser dashboard works across common phones and laptops without a dedicated installed app.
- The LCD provides an immediate address and concise status even without a browser.
- Configurable stops make the project reusable for different college routes.
- A simulation mode allows safe, repeatable indoor explanation of a complete bus journey.
- Route settings and trip summaries persist across ordinary restarts.
- Real GPS and simulated values are visually distinguished to avoid misleading demonstrations.
- Open-source tools and common components help reduce the cost of the prototype.

8.3 Limitations

- The dashboard is reachable only by devices on the ESP32 or associated local network; there is no public cloud tracking.
- The real GPS ETA uses straight-line distance and average/fallback speed, not real-time traffic or road travel time.
- Map tiles and road geometry require internet; an isolated hotspot may not display street-map imagery.
- GPS reception can degrade in indoor, obstructed or multipath conditions.
- A fixed 80-metre, eight-second arrival detector may not fit every road or bus speed.
- Only one bus is managed by the current embedded instance.
- The history stores up to thirty summaries, not long-term continuous location records.
- The software uses local HTTP and a shared administrator credential, which is not sufficient for public deployment.
- There is no standalone roadside passenger information display in the completed build.

8.4 Applications

- Demonstration of a college bus-monitoring concept in an ECE or IoT laboratory.
- Teaching GPS/NMEA decoding, serial communication, I2C and ESP32 networking.
- Explaining simple route-distance and arrival-time algorithms.

- Academic demonstrations of browser-based embedded monitoring.
- Foundation for a larger fleet, school bus or campus shuttle management prototype.
- Controlled transport simulations for workshops and student project exhibitions.

8.5 Privacy and Responsible Use

Location data can reveal movement patterns of vehicles and their occupants. Even in a mini-project, administrator passwords should not be embedded in publicly shared screenshots, reports or video descriptions. Use a trusted local network and avoid exposing actual passenger or operator information. Deployment beyond a classroom would require proper user authentication, HTTPS, access controls, data retention rules and consent practices.

8.6 Summary

The implemented BUSLYTICS design is useful as an educational local tracking prototype. Its principal development needs for practical service deployment are reliable wide-area connectivity, better prediction inputs and stronger security controls.

CHAPTER 9

CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

The Smart Bus Tracking and Arrival Prediction System demonstrates the integration of GPS location sensing, microcontroller processing, local Wi-Fi communication and an interactive passenger-oriented web dashboard. A NEO-6M GPS receiver supplies position and movement data to an ESP32, which manages the LCD, route progression, JSON interface and stored configuration. The browser application presents live status, stop-wise arrival estimates and visual monitoring, while a clearly labelled simulator supports controlled demonstration of bus journeys.

The final implementation achieves the main objectives of GPS acquisition, local visualization, route setup and approximate arrival prediction for one configured bus. Local flash-based trip summaries and controller diagnostics extend the project beyond basic GPS display. By combining multiple embedded and software disciplines in a single compact system, the prototype provides substantial practical learning value for Electronics and Communication Engineering students.

The implementation should be understood within its true scope. It does not include a public cloud backend, 4G/GSM module, real-time traffic-aware prediction or physical display board at a bus stop. Arrival predictions are approximate and depend on direct-distance estimates and recent or fallback speed. These limitations do not prevent the prototype from serving its educational purpose, but they define the work required for a production-scale service.

9.2 Future Scope

9.2.1 Cellular and Cloud Integration

A future version may use a suitable 4G communication device and a secured cloud backend to upload GPS positions while the bus moves outside the range of a local Wi-Fi network. A cloud database and passenger application could then support location viewing from remote stops, subject to appropriate network, billing and security arrangements.

9.2.2 Multi-Bus Fleet Monitoring

The local single-bus data model could be extended to multiple tracked vehicles with unique IDs, route assignment, operator management and fleet-level status screens. A server would coordinate updates rather than requiring each passenger to connect directly to a moving ESP32.

9.2.3 Improved Arrival-Time Prediction

Using road-network distance, recent trip history, time-of-day patterns, dwell-time information and live traffic feeds could improve ETA accuracy. Predictions should be validated against actual stop arrivals, and uncertainty should be communicated where measurement or traffic conditions are unreliable.

9.2.4 Passenger Notifications and Physical Stop Display

A mobile application, push notifications or an electronic roadside display could provide arrival information without a user repeatedly opening a dashboard. Such extensions would require independent network connectivity, backend publishing and appropriate mounting and power design.

9.2.5 Reliability and Security

Future improvements may include secure authentication, HTTPS or other encrypted channels, watchdog and diagnostic logging, resilient connectivity, configurable geofencing and durable journey storage during power interruptions. Vehicle installation would also need electrical and environmental protection.

9.3 Final Remarks

The completed BUSLYTICS prototype establishes a workable basis for more advanced intelligent transport applications. Its present value lies in demonstrating the connection between embedded GPS hardware, numerical estimation, persistent configuration and a usable browser interface within an economical local system.

REFERENCES

[1] K. Premkumar, K. Pavithra, J. Presiela, D. Priyadarshini and P. Priyanga, "College Bus Tracking and Notification System," International Conference on System, Computation, Automation and Networking (ICSCAN), 2020. Reference listed in the supplied Phase-1 presentation.

[2] S. Vigneshwaran, B. Nithya, K. Raghul, B. Nivas and V. M. Kishore, "Design of Bus Tracking and Fuel Monitoring System," International Conference on Advanced Computing and Communication Systems (ICACCS), 2020. Reference listed in the supplied Phase-1 presentation.

[3] Jisha R. C., A. Jyothindranath and L. S. Kumary, "IoT Based School Bus Tracking and Arrival Time Prediction," International Conference on Advances in Computing, Communications and Informatics (ICACCI), 2017. Reference listed in the supplied Phase-1 presentation.

[4] B. Dhivyabharathi, B. A. Kumar and L. Vanajakshi, "Real Time Bus Arrival Time Prediction System under Indian Traffic Condition," International Conference on Intelligent Transportation Engineering (ICITE), 2016. Reference listed in the supplied Phase-1 presentation.

[5] Espressif Systems, "ESP32 Series Datasheet" and ESP32 Arduino platform documentation, manufacturer and developer documentation.

[6] Mikal Hart, "TinyGPSPlus: NMEA parsing library for Arduino," public library documentation and source repository.

[7] Arduino, "Arduino IDE Documentation," programming, board setup and Serial Monitor instructions.

[8] Benoit Blanchon, "ArduinoJson Documentation," JSON serialization and deserialization for embedded systems.

[9] Leaflet contributors, "Leaflet Documentation," browser-based interactive map display.

[10] OpenStreetMap contributors, "OpenStreetMap," online mapping data and tile services.

APPENDIX A - HARDWARE CONNECTIONS

The following table summarizes the tested wiring configuration documented in the project firmware. Power supply voltage must be checked against the actual module labels before installation.

Connection	From	To
GPS serial	NEO-6M TX	ESP32 GPIO16 (RX2)
GPS ground	NEO-6M GND	ESP32/common GND
GPS supply	NEO-6M VCC	Compatible module supply
LCD data	I2C SDA	ESP32 GPIO21
LCD clock	I2C SCL	ESP32 GPIO22
LCD ground	GND	ESP32/common GND
LCD supply	VCC	Module-rated supply; level matching if needed
Web access	Wi-Fi	ESP32 AP or same-router LAN

A.1 Principal Electrical Notes

- Do not apply a voltage higher than the ESP32 GPIO tolerance to GPS or LCD data pins.
- Check the I2C backpack pull-up voltage if the LCD is powered at 5V.
- Use a regulated USB/board power input and common grounding.
- Keep the antenna in an open location for GPS acquisition.
- The source defines GPIO17 as UART2 TX but does not require a physical connection to GPS RX.

APPENDIX B - USER OPERATING PROCEDURE

B.1 Starting the System

24. Connect the prepared controller to its compatible regulated power supply.
25. Allow the GPS receiver time to acquire satellites in an open area.
26. Read the displayed IP address on the LCD.
27. Connect the viewing device to the ESP32 hotspot or the same Wi-Fi router.
28. Open the indicated address in a web browser.

B.2 Configuring a Route

29. Open Route Setup from the dashboard sidebar.
30. Enter a bus ID, route name and fallback speed.
31. Add stops in their correct travel sequence, providing latitude and longitude for each.
32. Set the next-stop index for the intended journey.
33. Enter the administrator password and save the configuration.
34. Confirm the configured route appears in Overview and Stops & ETA.

B.3 Demonstrating Real GPS

35. Select Overview or Live Tracking.
36. Verify Device Health reports a fresh real GPS fix.
37. Observe coordinates, speed, satellites and route information.
38. Compare current position with the configured next stop.
39. Keep the antenna and receiver outdoors if necessary for stable GPS acquisition.

B.4 Demonstrating Simulation

40. Configure at least two valid stops.
41. Open Bus Simulator and load the configured journey.
42. Choose speed and playback rate and start the virtual bus.
43. Demonstrate route movement and predicted stop arrival values.
44. Explain that values labelled SIM are virtual, not measured live bus positions.
45. Stop the simulation and return to Real GPS.

B.5 Recording a Trip

46. Open Trip History and supply administrator access.
47. For a real journey, select Start GPS Trip, travel with valid GPS reception and then Finish GPS Trip.
48. For a simulated journey, enable authenticated recording and complete the virtual route.
49. Refresh Trip History and inspect the resulting summary.
50. Use Delete or Clear History only when those records are no longer needed.

APPENDIX C - SYSTEM PARAMETERS AND API SUMMARY

Item	Setting
Microcontroller	ESP32 DevKit V1
GPS	NEO-6M; UART2 RX GPIO16; 9600 baud
I2C LCD	16 x 2 at address 0x27; SDA GPIO21; SCL GPIO22
Access point	Configured hotspot; default local IP 192.168.4.1
Web service	HTTP port 80
Max stops	12
Polling interval	2.5 seconds (browser)
GPS freshness	Less than 5 seconds
Geofence	80 metres / 8 continuous seconds
Saved trips	30 summaries
Demo speed	10 to 100 km/h
Simulation route	Online road routing with direct-line fallback
Cloud / remote web access	Not implemented in current version

C.1 Project Scope Declaration

This report describes the BUSLYTICS local edition as implemented in the supplied ESP32 source code. The experimental results chapter intentionally avoids unverifiable measurement values. Before academic submission, actual photographs, screenshots and field-observation records may be added as documentary evidence without changing the technical description.